VitiDrone FVG

Developer Guide — Analisi Tecnica Approfondita

Senior Lead AI Engineer Review · Computer Vision & Edge Computing · Marzo 2026

Versione	**2.0 — DJI Edition**
Focus	Architettura AI, librerie Python, pipeline inferenza edge
Hardware target	Jetson Orin Nano 8GB · Raspberry Pi 4B · MacBook (MPS)
Classificazione	*Riservato — Solo uso interno*

1. Architettura dei Modelli e Pipeline di Inferenza

Il report definisce una pipeline a tre stadi distinti per complessità computazionale crescente: pre-screening leggero (YOLOv8-nano), detection d'istanza fine-grained (Mask R-CNN), e classificazione patologica profonda (ResNet-50 fine-tuned). Ogni modello è assegnato a un agente specifico della flotta.

1.1 YOLOv8-nano — Agente Scout / Test Amatoriale

Contesto operativo: Workhorse della fase zero (DJI Mini 3 + Mac) e del sistema Scout professionale (Matrice 30T). Opera in modalità object detection con bounding box, ottimizzato per latenza minima su hardware consumer.

Metodologia: Single-stage detector basato su architettura CSP (Cross Stage Partial) con head decoupled. L'architettura "nano" ha ~3.2M parametri (vs 11.2M di YOLOv8s), sacrificando recall su classi sottorappresentate — rilevante per patologie rare come Flavescenza dorata vs peronospora.

Hardware target e ottimizzazione:

Contesto	Hardware	Framework	FPS atteso
Test amatoriale (Opzione A/B)	MacBook (CPU/MPS)	PyTorch → ONNX	25–30 FPS
Opzione C campo	Raspberry Pi 4B (4GB)	PyTorch FP32 ARM	8–12 FPS
Scout professionale	Jetson Orin Nano 8GB (40 TOPS)	TensorRT FP16	60–90 FPS est.

Benchmark citati nel report:

- Accuracy AI su Mini 3 (RGB only, no multispettrale): **78–83%** — assenza di bande NIR/RedEdge
- Latenza end-to-end stream live (RTMP, Opzione B): **150–350 ms** — accettabile per validazione
- Latenza stream Opzione C (RTSP via CosmoStreamer): **<200 ms** a 1080p/60fps

Note architetturale critica: *Il collo di bottiglia non è l'inferenza YOLOv8-nano (<5ms su Jetson), ma il pipeline di trasporto video: O3+ link → RC-N1 → USB-C decode → stream encode → rete locale. Questo introduce una latency stack di 120–280ms prima ancora che il frame raggiunga il modello.*

1.2 ResNet-50 Fine-Tuned — Agente Analyzer (Core Classifier)

Contesto operativo: Montato su Jetson Orin Nano 8GB via DJI Payload SDK (interfaccia USB). Opera sul feed Zenmuse H30T a 5 bande multispettrali + termico — input ben oltre i 3 canali RGB standard.

Metodologia (workflow a 3 step):

- YOLOv8 localizza la Region of Interest (ROI) fogliare
- Il crop della ROI viene passato a ResNet-50 per classificazione fine-grained della patologia
- Output: classe patologica + confidence score geolocalizzato RTK ±2cm

Adattamento multi-banda: ResNet-50 nativamente accetta tensori [B, 3, H, W]. Per le 5 bande H30T (Blue, Green, Red, RedEdge, NIR) + termico è necessario un layer Conv2d di ingresso personalizzato con 6 canali. Il fine-tuning deve avvenire su dataset viticolo FVG specifico (peronospora, oidio, botrite, Flavescenza dorata).

Conversione PyTorch → ONNX → TensorRT per Jetson Orin Nano:

```
import torch, tensorrt as trt

model = ResNet50MultiSpectral(num_classes=8, in_channels=6)
model.load_state_dict(torch.load('vitidrone_resnet50_fvg_v1.pth'))
model.eval()

dummy_input = torch.randn(1, 6, 224, 224)
torch.onnx.export(model, dummy_input, "vitidrone_analyzer.onnx",
                  opset_version=17, dynamic_axes={'input': {0: 'batch_size'}})

TRT_LOGGER = trt.Logger(trt.Logger.WARNING)
with trt.Builder(TRT_LOGGER) as builder:
    config = builder.create_builder_config()
    config.set_flag(trt.BuilderFlag.FP16)   # ~2x speedup su Orin Nano
    config.set_memory_pool_limit(trt.MemoryPoolType.WORKSPACE, 1 << 30)
```

Benchmark: Inferenza complessiva <30ms (include pre-processing MS + ResNet forward pass su TRT FP16). Accuracy sistema completo con H30T: **>92%**.

1.3 Mask R-CNN — Stima Resa (Conteggio Grappoli)

Contesto operativo: Non fa parte del sistema di rilevamento patologie, ma della use case #02 (stima vendemmia). Opera sul Matrice 30T con camera zoom 200x a GSD <5mm/px.

Metodologia — Instance segmentation: A differenza di YOLOv8 (bounding box), Mask R-CNN genera maschere pixel-level per ogni istanza di grappolo. La stima del peso usa la superficie della maschera come proxy volumetrico. Due output paralleli: RPN (Region Proposal Network) per localizzazione + branch di segmentazione per conteggio.

Vincolo critico: ~60M parametri, ~3.5GB VRAM FP16. Non può co-esistere con ResNet-50 su Jetson Orin Nano 8GB senza scheduling esplicito. Benchmark: errore stima resa **<8%** (vs ±25–35% manuale).

2. Stack Librerie e Dipendenze Python

Libreria	Layer	Ruolo nella pipeline
`ultralytics >= 8.0.0`	Core Vision	YOLOv8: training, export multi-formato (ONNX/TRT/CoreML), inferenza one-liner. Supporta export per Jetson via model.export(format='engine').

`opencv-python >= 4.9.0`	Video Stream	Gestisce buffer VideoCapture per RTSP/RTMP, decode H.264/H.265, conversione colorspace. Su RPi4B richiede build con GStreamer abilitato.
`pyexiftool >= 0.5.6`	Geo Metadata	Estrae lat/lon/alt RTK, orientamento gimbal e parametri fotocamera dai tag XMP/EXIF di ogni immagine DJI. Sorgente primaria per la mappa dei focolai.
`rasterio >= 1.3.0`	Multispettrale	I/O GeoTIFF a 5 bande Zenmuse H30T. Necessario perché OpenCV non supporta nativo il formato multibanda geospaziale.
`fastapi >= 0.110.0`	Backend	Implementa il Commander come microservizio cloud async. Supporto nativo async/await — critico per la gestione concorrente di N stream drone simultanei.
`torch >= 2.2.0`	Training	Backend PyTorch per fine-tuning ResNet-50 (6ch input) e Mask R-CNN. Esportazione verso ONNX poi TRT per deployment su Jetson.
`boto3 >= 1.34.0`	Cloud	AWS IoT + S3 per storage missioni e telemetria. Connette il Commander cloud al layer di persistenza dei dati.
`openai >= 1.12.0`	AI Agronomico	GPT-4o-mini API (RAG su knowledge base FVG) per traduzione dati tecnici in raccomandazioni agronomiche in linguaggio naturale.

3. Implementazione e Protocolli di Streaming

3.1 Opzione A — Analisi Offline microSD (€0, immediata)

Flusso dati: DJI Mini 3 → microSD → Mac → Python. Nessuna latenza di rete, massima qualità immagine (RAW+JPEG), georeferenziazione completa via EXIF. Limite: nessun feedback in tempo reale durante il volo.

```
# pipeline_offline_optionA.py
from pathlib import Path
import exiftool, pandas as pd
from ultralytics import YOLO

IMAGES_DIR = Path("/Volumes/DJI_MINI3/DCIM/100MEDIA")
MODEL_PATH  = "vitidrone_yolov8n_diseases_v1.pt"

def extract_gps_metadata(image_paths):
    with exiftool.ExifToolHelper() as et:
        metadata = et.get_tags([str(p) for p in image_paths],
            tags=["GPS:GPSLatitude","GPS:GPSLongitude",
                  "XMP:RelativeAltitude","XMP:GimbalPitchDegree"])
    return {m["SourceFile"]: m for m in metadata}

def run_inference_batch(images_dir, model_path):
    model   = YOLO(model_path)
    results = model.predict(source=str(images_dir), conf=0.45,
```

```
                            imgsz=640, stream=True, verbose=False)
    gps_data = extract_gps_metadata(list(images_dir.glob("*.JPG")))
    records  = []
    for result in results:
        gps = gps_data.get(result.path, {})
        for box in result.boxes:
            records.append({
                "image":      Path(result.path).name,
                "latitude":   gps.get("GPS:GPSLatitude"),
                "longitude":  gps.get("GPS:GPSLongitude"),
                "altitude_m": gps.get("XMP:RelativeAltitude"),
                "class_name": result.names[int(box.cls)],
                "confidence": float(box.conf),
            })
    return pd.DataFrame(records)

if __name__ == "__main__":
    df = run_inference_batch(IMAGES_DIR, MODEL_PATH)
    df.to_csv("output/focolai_mappa.csv", index=False)
    print(f"Rilevamenti salvati: {len(df)}")
```

3.2 Opzione B — Stream Live RTMP via MSDK v5 (~€130)

Flusso dati: Mini 3 → O3+ → RC-N1 → USB-C → Android (MSDK v5) → RTMP → Mac (OpenCV) → YOLOv8. Latenza end-to-end: 150–350ms. Il bottleneck è il percorso O3+ wireless → USB decode Android → encode RTMP → rete Wi-Fi.

```
# pipeline_live_rtmp_optionB.py
import cv2, time
from ultralytics import YOLO

RTMP_URL = "rtmp://192.168.0.105:1935/live/dji_stream"

def create_rtmp_capture(url):
    cap = cv2.VideoCapture(url, cv2.CAP_FFMPEG)
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)   # Riduce buffer → minimizza latenza
    return cap

def inference_loop(cap, model):
    fps_counter, t0 = 0, time.time()
    while True:
        ret, frame = cap.read()
        if not ret:
            time.sleep(0.1); continue
        results = model.predict(frame, conf=0.45, device="mps", verbose=False)
        fps_counter += 1
        if fps_counter % 30 == 0:
            elapsed = time.time() - t0
            print(f"FPS: {fps_counter/elapsed:.1f} | Latenza: {1000/max(fps_counter/elapsed,1):.0f}ms")
        cv2.imshow("VitiDrone Scout", results[0].plot())
        if cv2.waitKey(1) & 0xFF == ord('q'): break

model = YOLO("vitidrone_yolov8n_diseases_v1.pt")
cap   = create_rtmp_capture(RTMP_URL)
try:    inference_loop(cap, model)
finally: cap.release(); cv2.destroyAllWindows()
```

3.3 Opzione C — RTSP via CosmoStreamer + Raspberry Pi 4B (~€165, raccomandata)

Flusso dati: Mini 3 → O3+ → RC-N1 → USB-C → RPi4B (CosmoStreamer) → RTSP → Mac/Jetson → YOLOv8. Latenza: <200ms a 1080p/60fps. Il FrameBuffer thread-safe evita il frame pile-up (inferenza più lenta della cattura → analisi sempre sul frame più recente).

```
# pipeline_live_rtsp_optionC.py
import cv2, threading, queue
from ultralytics import YOLO

RTSP_URL = "rtsp://192.168.1.42:8554/stream"

class FrameBuffer:
    # Buffer thread-safe: mantiene sempre l ultimo frame disponibile
    def __init__(self): self.q = queue.Queue(maxsize=2)
    def put(self, f):
        if self.q.full(): self.q.get_nowait()
        self.q.put(f)
    def get(self): return self.q.get(timeout=1.0)

def capture_thread(url, buffer):
    cap = cv2.VideoCapture(url, cv2.CAP_GSTREAMER)
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
    while True:
        ret, frame = cap.read()
        if ret: buffer.put(frame)

model  = YOLO("vitidrone_yolov8n_diseases_v1.pt")
buffer = FrameBuffer()
t = threading.Thread(target=capture_thread, args=(RTSP_URL, buffer), daemon=True)
t.start()
while True:
    try: frame = buffer.get()
    except queue.Empty: continue
    results = model.predict(frame, conf=0.45, verbose=False)
    cv2.imshow("VitiDrone RTSP", results[0].plot())
    if cv2.waitKey(1) & 0xFF == ord('q'): break
```

3.4 Model Scheduler — Jetson Orin Nano 8GB (Flotta Pro)

Mask R-CNN (~3.5GB FP16) e ResNet-50 (~1.2GB FP16) non possono coesistere. Il ModelScheduler implementa lazy loading con `torch.cuda.empty_cache()` tra i modelli. YOLOv8-nano (~180MB) rimane sempre warm per lo screening continuo.

```
# model_scheduler.py — Jetson Orin Nano on-board
from enum import Enum
import torch

class ActiveModel(Enum):
    NONE = "none"; ANALYZER = "resnet50"; HARVEST = "maskrcnn"

class ModelScheduler:
    def __init__(self):
        self.active   = ActiveModel.NONE
        self.resnet   = None; self.maskrcnn = None
        from ultralytics import YOLO
        self.yolo = YOLO("vitidrone_yolov8n.engine")  # sempre warm ~180MB

    def switch_to_analyzer(self):
        if self.active == ActiveModel.ANALYZER: return
        if self.maskrcnn:
            del self.maskrcnn; torch.cuda.empty_cache(); self.maskrcnn = None
        self.resnet = self._load_trt("analyzer_resnet50_fp16.engine")
```

```
        self.active = ActiveModel.ANALYZER

    def switch_to_harvest(self):
        if self.active == ActiveModel.HARVEST: return
        if self.resnet:
            del self.resnet; torch.cuda.empty_cache(); self.resnet = None
        self.maskrcnn = self._load_trt("harvest_maskrcnn_fp16.engine")
        self.active = ActiveModel.HARVEST

    def _load_trt(self, path):
        import tensorrt as trt
        rt = trt.Runtime(trt.Logger(trt.Logger.ERROR))
        with open(f"/opt/vitidrone/{path}", "rb") as f:
            return rt.deserialize_cuda_engine(f.read())
```

4. Matrice Comparativa Tecnica dei Modelli

Modello	Task	Hardware	Parametri	VRAM est.	Accuracy	FPS	Latenza	Agente
YOLOv8-nano (Mac MPS)	Detection	MacBook PyTorch	3.2M	~200MB	78–83%	25–30	5ms+ 150–350ms	Scout/ Test
YOLOv8-nano (RPi 4B)	Detection	ARM A72 FP32	3.2M	~350MB RAM	78–83%	8–12	80–125ms	Scout campo
YOLOv8-nano (Jetson TRT)	Detection	Orin Nano TRT FP16	3.2M	~180MB	78–83%	60–90	<5ms	Scout M30T
ResNet-50 Fine-tuned	Classif. patch	Orin Nano TRT FP16	25M	~1.2GB	>92% (H30T)	30–40	<30ms	Analyzer M350
Mask R-CNN grappoli	Instance Segm.	Orin Nano TRT FP16	60M	~3.5GB	<8% errore resa	8–12	80–125ms	M30T resa

Nota schedulazione: *Su Jetson Orin Nano 8GB, Mask R-CNN e ResNet-50 non possono coesistere. YOLOv8-nano rimane sempre warm (~180MB). Il ModelScheduler gestisce il lazy loading con torch.cuda.empty_cache() tra switch di modello.*

5. Riepilogo Architetturale

Layer di trasporto	`O3+ → RC → USB → CosmoStreamer/MSDK → RTSP/RTMP`	Vero collo di bottiglia per la latenza end-to-end, non l'inferenza AI. Il frame pile-up e la latency stack di trasporto (120–280ms) dominano l'end-to-end.
Layer di inferenza	`YOLOv8-nano → ResNet-50 → Mask R-CNN`	Scala dall'hardware consumer all'edge AI industriale con architettura modulare. Il layer hardware rimane intercambiabile grazie all'export ONNX intermedio.

Layer di orchestrazione	`FastAPI Commander → FlightHub 2 API → MSDK multi-agente`	Componente più differenziante. La coordinazione cloud-side su 4G risolve l'assenza di comunicazione drone-to-drone nativa DJI, trasformando droni consumer in agenti di un sistema distribuito.

© 2026 VitiDrone FVG — Startup Accademica, Università FVG | Developer Guide v2.0 — DJI Edition | Documento interno riservato